WWW.IJRPR.COM

International Journal of Research Publication and Reviews

Journal homepage: www.ijrpr.com ISSN 2582-7421

NAMI: Adaptive Intelligence for SLAM-Based Autonomous Indoor Navigation

Vivek Prakash[1], Ritik Pandey[2], Harish Mittal[3]

Department of Computer Science Engineering (AI)
IIMT College of Engineering, Greater Noida, India

ABSTRACT—

In this paper, we propose NAMI (Navigational Autonomous Mapping Intelligence) - an autonomous mobile robot system based on simultaneous localization and mapping in GPS-denied indoor environments. Our proposed approach uses multi-sensor fusion using 2D LiDAR, Inertial Measurement Units, and wheel odometry in a graph-based SLAM method for obtaining robust positioning and map construction. NAMI uses a differentially driven robot with YDLIDAR X2, MPU-6050 IMU, and optical encoders. It is operated under ROS Noetic with embedded computing using Raspberry Pi 4B microcontroller board. In this work, we address various issues related to autonomous navigation indoors such as sensor integration and calibration, pose-graph optimization, dealing with dynamic obstacles and computing limitations. We conduct experiments with NAMI for autonomous navigation indoors, where the robot successfully performs navigation between two points with sufficient localization accuracy for logistics tasks. Occupancy grid maps produced by the robot have clear geometry and consistency after long-term operation due to efficient loop-closure and trajectory correction methods. This paper presents the realization of autonomous navigation between two points indoors using classical SLAM algorithms implemented on embedded systems.

Keywords— autonomous navigation, SLAM, mobile robotics, sensor fusion, indoor mapping, ROS

I. INTRODUCTION

Self-navigation without any external support for position tracking is one of the central tasks for autonomous mobile robots operating indoors. As opposed to outdoor robots that can use GPS to estimate the absolute position, indoor robots suffer from the attenuation of GPS signals and their multipath nature, which makes it unreliable for such applications. Therefore, a self-sustained system is required, able to perform both localization and mapping in an unknown environment.

The Simultaneous Localization And Mapping (SLAM) algorithm aims at estimating the robot trajectory and mapping the structure of the environment based on the measurements produced by sensors. Such an approach represents a classic example of a chicken-and-egg problem requiring estimation and updating of the robot pose and landmarks positions at the same time. Contemporary SLAM algorithms provide sufficient development to work reliably in different environments and form the basis of all modern autonomous cars, service robots, and logistics.

In this paper, we introduce NAMI (Navigational Autonomous Mapping Intelligence), a fully functional autonomous navigation system designed for logistics applications within indoor settings. Our study confirms the capabilities of classical graph-based SLAM techniques when applied and tuned in the proper way, providing reliable results with embedded computing hardware without requiring any additional GPU processing power or costly sensor suites.

The following contributions have been made through our research efforts: (1) full system integration using affordable commercially available sensors combined with open-source SLAM algorithms; (2) calibration process and parameter tuning; (3) experimental evaluation of the system within real-world office scenarios, confirming its navigation abilities, and (4) performance assessment and identification of failure modes that may help enhance the current system in the future.

A. System Overview and Architecture

NAMI is based on a differential drive mobile platform with three complementary sensing modalities. The environment perception relies on 360° laser range measurement with a YDLIDAR X2. High frequency orientation estimates come from an MPU-6050 inertial measurement unit. Optical encoders on the drive motors are used to track the odometric position. These sensors are fed into an embedded computer, a Raspberry Pi 4B running a containerized ROS2.

The software architecture is based on modular components for sensor processing, localization, mapping, path planning and motor control. The SLAM Toolbox package implements graph-based SLAM . It maintains a pose graph that relates the robot poses with odometry constraints and scan-matching constraints . Loop closure detection finds previously visited places, allowing for global trajectory optimization to fix accumulated drift. Navigation is implemented using A* global path planning and Dynamic Window Approach for local path planning. It offers a good balance between computation time and path planning capability. All computations are performed by the onboard ARM microprocessor, and no GPU is involved. Thus, its feasibility has

been proved in low-cost applications.

B. Paper Organization

The hardware platform is discussed in Section II, which includes information about sensors, electronics, and mechanics. Software development is outlined in Section III, which discusses SLAM algorithms and sensor calibration methods. Experimental results are provided in Section IV, which includes maps created and navigation measures. System weaknesses and failure mechanisms are explained in Section V. The conclusions are stated in Section VI.

II. HARDWARE PLATFORM AND SENSOR INTEGRATION

The hardware platform of NAMI consists of commercially available off-the-shelf parts, chosen based on their availability, ROS compatibility, and performance requirements.

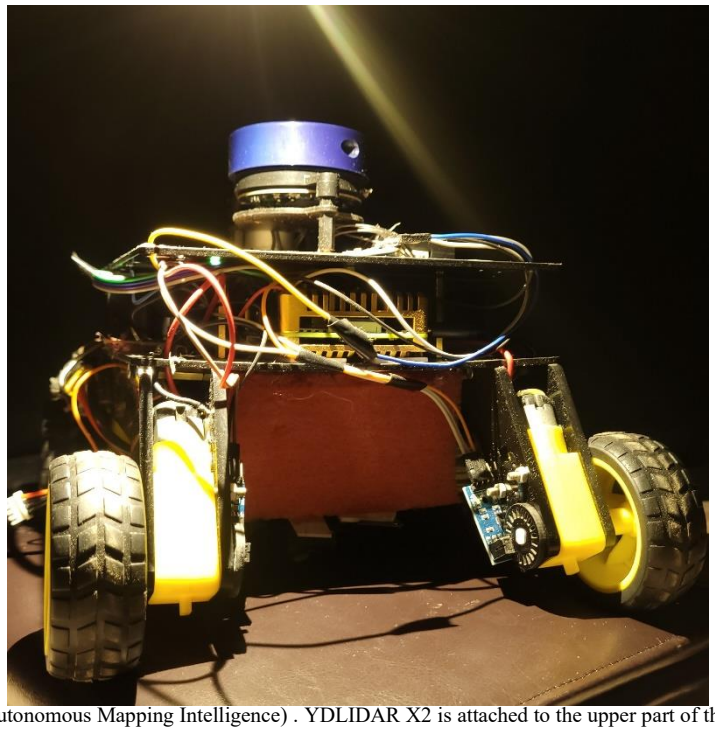

Figure 1. NAMI (Navigational Autonomous Mapping Intelligence) . YDLIDAR X2 is attached to the upper part of the platform to ensure unobstructed 360° field of view. Raspberry Pi 4B and motors controller are placed at the middle level of the robot platform.

A. Sensor Suite and Specifications

Hardware Specifications are listed in Table I. The YDLIDAR X2 sensor is capable of measuring distances at 360-degree angles with an accuracy of 0.45 degrees, an operational range of 8 to 12 meters, and a scanning speed of 6 to 7 hertz.

TABLE I

HARDWARE COMPONENT SPECIFICATIONS

Component	**Model/Type**	**Key Specifications**
Processor	Raspberry Pi 4B	Quad-core ARM Cortex-A72 @ 1.5GHz, 4GB RAM
LiDAR Sensor	YDLIDAR X2	360° coverage, 8-12m range, 0.45° angular resolution, 10kHz sample rate, 6-7Hz scan frequency
IMU	MPU-6050	6-axis (3-axis accelerometer + 3-axis gyroscope), 100Hz update rate, onboard DMP sensor fusion
Wheel Encoders	Optical coupling	100 pulses per revolution, 5mm theoretical resolution
Drive Motors	DC brushed gearmotor	125 RPM no-load speed, 12V nominal, dual-shaft configuration

Power Supply	LiPo battery pack	11.1V nominal (3S configuration), 5200mAh capacity, 57.72Wh total energy, 35C discharge rating
Chassis	Differential drive	35cm × 30cm × 20cm (L×W×H), 5cm ground clearance, 9cm wheel diameter, 20cm wheelbase

The MPU-6050 inertial measurement device incorporates a 3-axis accelerometer together with a 3-axis gyroscope, both coupled with a Digital Motion Processor on board. The DMP computes sensor fusion at a rate of 100Hz in order to relieve the load on the main processor. The I2C communication runs at a speed of 400kHz.

The optical encoders that are attached to the shaft of the motor generate 100 pulses per revolution, providing a resolution of 5mm odometry. Despite being less effective in comparison to industrial optical encoders with 1000+ pulses per revolution, it is sufficiently accurate following calibration. The encoders receive impulses on the input pins of the Raspberry Pi GPIO.

The Raspberry Pi 4B possesses enough computing power to perform SLAM computation in real-time without requiring a dedicated graphics processing unit. The quad core ARM Cortex-A72 processor processes sensor fusion, scan matching and path finding computations while keeping control of the motors under check.

B. Electronic Architecture and Power Distribution

Figure 2 depicts the entire electronic circuitry. The system uses only one 11.1V LiPo battery, which powers all systems via regulated power supply. A buck regulator reduces battery voltage to 5V/3A to supply the

Raspberry Pi, which runs stably despite variations in battery voltage due to discharge (12.6V when charged to 9.0V when discharged).

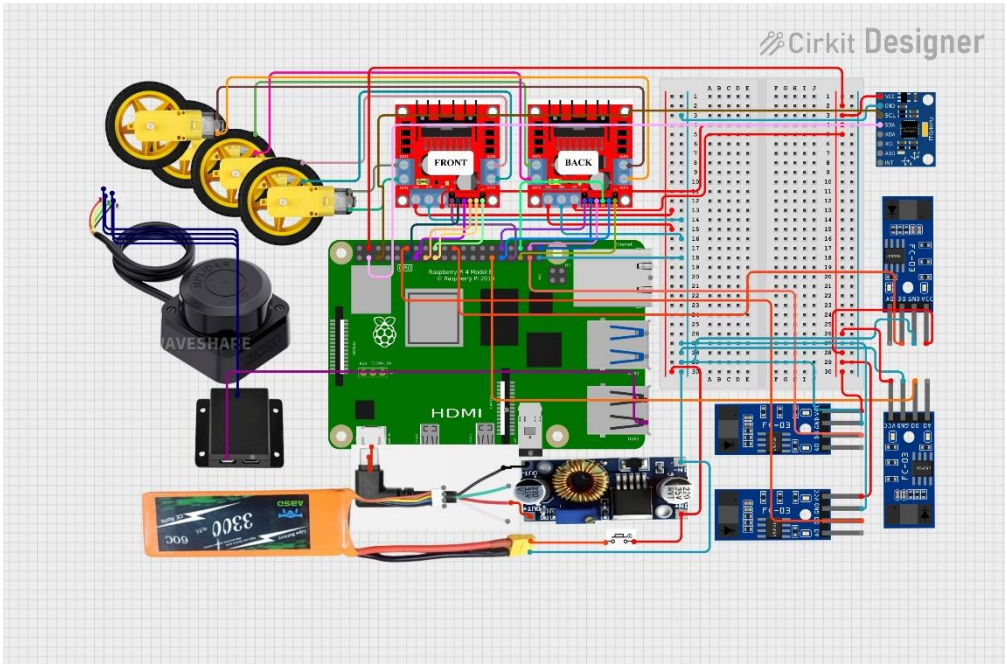

Figure. 2. *Electronic Architecture of the System*, indicating the incorporation of the Raspberry Pi 4B, YDLIDAR X2, motor drivers, power, and sensor module

Motor control makes use of an L298N H-bridge driver that provides differential drive control by allowing current flow in both directions. The driver receives PWM pulses from the Raspberry Pi GPIO pins to control the speed, and digital logic pulses for controlling the direction of the motor. Individual enable lines for each wheel are used to independently vary the speed, which is necessary for trajectory tracking.

The YDLIDAR X2 sensor is connected through USB, acting as a virtual COM port for the ROS driver. The MPU-6050 IMU sensor communicates using the I2C bus (GPIO 2 & 3), with a maximum clock frequency of 400kHz for acquiring 100 Hz readings. The encoder pulses are connected to GPIO pins 17, 18, 22, and 23 with interrupts enabled.

C. Mechanical Design Considerations

The frame design uses a hierarchical structure that positions components based on center-of-gravity considerations and the requirement for clear LiDAR scanning. The battery is placed at the bottom layer to reduce the system's center of gravity for better stability when performing fast maneuvers. The Raspberry Pi and motor driver are placed at the second layer, and the LiDAR sensor is located at the third layer.

The LiDAR mounting height greatly impacts the accuracy of the scan data collected. Placing the sensor at 25cm from the ground level ensures that the LiDAR scans below typical table levels (70-75cm), while it remains above obstacles at ground level. At this height, the LiDAR scan plane passes through the widest part of any standing obstacle.

The differential drive system comprises two wheels of 9 cm diameter with a distance between wheels of 20 cm. According to calculations, the minimum turning radius is 10cm (half wheelbase), which means that the robot will be able to move through ordinary door openings, with the minimum width of 80 cm minus the width of the robot (30 cm).

The vibration isolation of the LiDAR sensor helps prevent motor-generated vibrations from interfering with the scanning process. The foam dampers used on the LiDAR mount help absorb the vibrations between 50Hz and 200Hz frequencies resulting from motor movements. This was important since consistent wall detection required vibration isolation.

III. SOFTWARE ARCHITECTURE AND SLAM IMPLEMENTATION

The navigation package uses a module-based design architecture under the Robot Operating System structure. This allows for standardized communication between processes, isolation from other modules, and usage of well-established SLAM and navigation algorithms.

A. Software Architecture and Component Interfaces

The system comprises five primary ROS nodes: (1) YDLIDAR driver publishing LaserScan messages at 6-7Hz, (2) encoder odometry node computing pose increments from wheel rotation at 100Hz, (3) SLAM Toolbox maintaining the pose graph and occupancy grid, (4) move_base navigation stack for path planning and trajectory control, and (5) motor control node translating velocity commands to PWM signals.

Data flow follows a sensor-processing-actuation pipeline. The YDLIDAR driver reads the raw range data and publishes to /scan topic. The encoder odometry node integrates the wheel rotation counts to Odometry messages using differential drive kinematics and publishes to /odom. SLAM Toolbox subscribes to both topics and fuses scan matching constraints with odometry constraints to estimate robot pose and build the map.

The navigation stack computes velocity commands from the current map, robot pose and goal location. The global planner uses an A* search over the occupancy grid to generate collision-free paths. Dynamic Window Approach is used as the local planner to generate velocity commands to follow the global path and avoid the dynamic obstacles. These commands are sent to the motor control node, which converts the desired linear and angular velocity into differential speeds for the wheels.

Startup sequencing ensures correct initialization dependencies. The encoder odometry node should initialize first to create a stable odometry frame for other nodes to subscribe to. The SLAM Toolbox is the second to start, and needs valid odometry data before starting scan matching. The navigation stack is the last to be switched on, once the SLAM system has built an initial map. This ordered startup avoids race conditions that could result in localization failures.

B. Graph-Based SLAM Configuration

NAMI utilizes SLAM Toolbox, a graph-based SLAM with pose-graph optimization. The algorithm maintains a graph whose nodes correspond to robot poses at discrete times, and whose edges correspond to spatial constraints between poses. Odometry measurements give rise to sequential edges between consecutive poses. In case of the current observations matching with previously mapped areas scan matching produces non-sequential edges which can be used to detect loop closures .

Systematic experimentation revealed critical configuration parameters. The grid is configured to have 5 centimeters resolution per cell, which provides sufficient spatial accuracy for obstacle avoidance while limiting memory consumption. For instance, a grid of 200 by 200 cells requires about 40 kilobytes of memory. Finer resolution will be more computationally expensive but will not enhance navigation performance for a 30cm-wide robot.

To find loop closures, scan correlation is used with a search radius of 10 meters and a correlation threshold of 0.85. The search radius keeps the cost of computing low by only allowing candidate matches to be found in historical poses that are close by. The correlation threshold keeps the rates of false positives and false negatives in check. Lower values cause false loop closures in environments that repeat; higher values miss valid closures that could fix drift.

Scan matching parameters determine the balance between accuracy and strength. Even with wheel slip and odometry drift, matching is still possible with a maximum correspondence distance of 0.2 meters. The covariance scaling factors give more weight to scan matching constraints than odometry constraints (0.7:0.3 ratio). This shows that LiDAR measurements are more reliable than encoders on smooth indoor floors where wheel slip happens.

C. Sensor Calibration Methodology

Odometry relies on proper calibration of the kinematic model parameters. Mechanical measurement of the wheel diameter gave a value of 9.0 cm, whereas the actual diameter due to deforming tire during motion was found to be 8.95 cm. In one test for calibration with an accuracy measurement from outside the robot on a straight path of 10 meters, the odometry system suffered from a systematic error of 2.3%, which was corrected using a scaling factor for the number of encoder pulses.

For calibration of the wheelbase, a 360-degree turn was required to be commanded and compared between measurements by the IMU and the encoder.

The correction needed due to wheel slip when turning made the mechanical wheelbase value of 20.0 cm to an effective value of 19.4 cm.
The IMU calibration took place using several steps. The accelerometer bias value was calculated while the robot stood still with the help of 1000 samples being averaged to find zero-g offsets for all axes. Gyro bias measurement was done during 60 seconds of standstill; gyro drift rate amounts to approximately 0.8 degrees per second. Those biases are then subtracted from readings before integration is done. The magnetometer is not used since it experiences strong magnetic disturbances caused by motors and indoor environment.
The main issue with LiDAR calibration was checking angular offset alignment. According to manufacturer specifications, zero degrees of scan should coincide with forward-facing side of LiDAR; those sides were mechanically aligned to be oriented toward the robot's front side. The calibration was carried out when a target object (known wall) was located directly ahead of the robot, and zero degrees corresponded to the closest-point-of-scan line.

IV. EXPERIMENTAL VALIDATION AND RESULTS

The validation of the system took place by means of field testing in actual indoor settings. The purpose of the experiments was to validate the system's ability to navigate autonomously and analyze its performance under various environmental settings.

A. Experimental Setup and Environmental Settings

The experiments were performed in two main types of environments: structured hallways of offices and complex rooms having different degrees of obstacles. Figure 3 below illustrates one of the test environments, which is an office hallway of a university with glass walls, doorways, and tiled floors.

Figure 3. *Testing environment where the NAMI is deployed within an ordinary office hallway.* The testing environment contains glass walls, tiles flooring, and changing light conditions.

The testing process included autonomous exploration for mapping purposes and autonomous navigation. At first, the robot would explore its surroundings and generate an occupancy grid map of the area, after which it would complete various navigation tasks based on ROS commands.

B. Occupancy Grid Mapping Results

Occupancy grids of **Figures** 4 and 5 were constructed during the autonomous navigation phase. As per the usual conventions, black squares symbolize obstacles, white squares denote free space, and gray squares mark unknown areas.

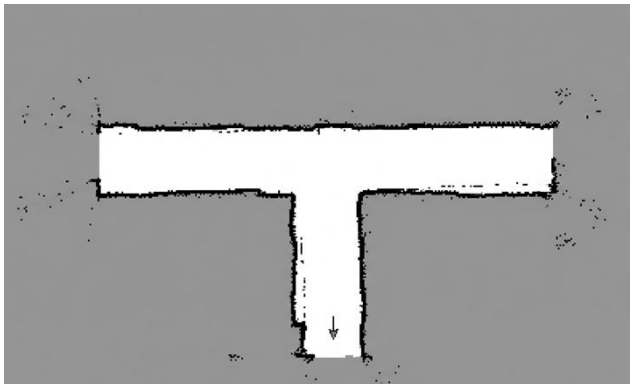

Figure 4. *Occupancy Grid Map of the structured corridor/office space*. This map indicates a corridor like shape with a well-defined T-junction, showing efficient SLAM and high geometrical accuracy.
Figure 4 represents the performance mapping on an indoor structured corridor. The walls in the map are almost continuous with high sharpness to its layout. Small distortions in the edges of the walls and presence of some scattered specks around the free space can be observed, showing minimal noise in the sensor reading but still making the layout quite useable .

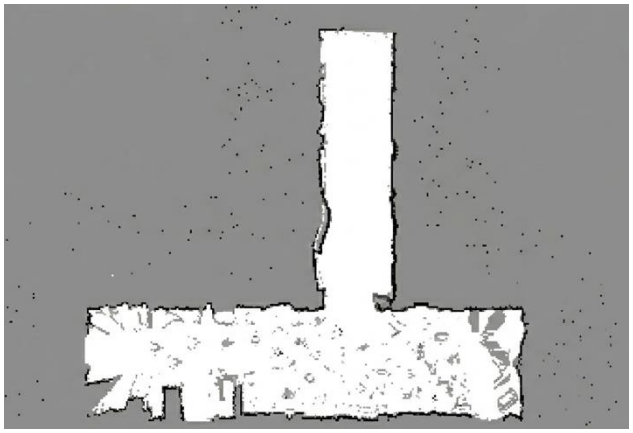

Figure. 5. Map of apartment environment with higher noise and clutter. Grid map of the apartment-like environment with higher noise and clutter.

Figure 5 demonstrates mapping for an indoor setting with greater challenges. The floorplan is similar, but the added noise is indicative of dynamic interference, clutter, and less structure compared to Figure 4. Nevertheless, the grid map retains the underlying geometry of the apartment, specifically the corridor-like space in the middle and adjacent rooms, suggesting that mapping is successful despite lower sensor accuracy. The higher presence of obstacles means that the resulting occupancy map is less clean than that generated by mapping in the hallway setting.

It is clear that the algorithm produces better results in structured environments due to geometric limitations leading to cleaner maps. Nevertheless, it is sufficient for autonomous navigation indoors.

C. Autonomous Navigation Performance Analysis

For autonomous navigation tests, at least 50 trials were performed in which goals were set, the robot independently navigated, avoided obstacles, and arrived to destinations. Successful performance is considered an arrival point inside 20cm from the goal position without collision and without help from humans.

The percentage success was higher for experiments carried out in the environment similar to a corridor or office space than in the environment similar to an apartment. Navigation in the former environment was easier due to the ability to use scan matching better as a result of more structured space and clearer intersections.

There were several reasons for failures during experiments. Firstly, the problem might be related to the difficulty of tracking localization. As in the second environment, there were more factors disturbing proper scan matching such as more cluttered area or noisiness. Secondly, collisions could occur more frequently in environments similar to apartments than in corridors or offices because the former type had more narrow passages and partially occupied zones. Finally, there could be difficulties with navigation because of the presence of noise in occupancy update in the apartment environment, resulting in creating smaller false obstacles for further motion.

In summary, the experiments confirm that simple and inexpensive sensors along with traditional SLAM techniques enable the creation of effective maps for autonomous navigation. The map of the hallway is more precise in terms of occupancy detection, whereas the apartment map reveals that SLAM systems continue functioning but become more susceptible to noise.

V. PERFORMANCE ANALYSIS AND LIMITATIONS

Analysis of experiment results shows the strengths and weaknesses of the current implementation.

A. Computational Performance Characteristics

CPU usage varied at around 35% in steady-state motion and went up to 62% during optimization of loop closures. Memory usage was consistently kept under 180MB for the entire stack of navigation algorithms running, which is comfortably below the 4GB memory on board of Raspberry Pi 4B. This shows that real-time performance has been achieved without using any GPU accelerations.

The latency of computation from scanning by LiDAR to updating pose was 42 milliseconds, or equivalently, the rate of computation was 24Hz, which exceeded the 6-7Hz sampling rate of LiDAR, thus no loss of data. The end-to-end latency from sensing through decision making to actuation averaged at 78 milliseconds, allowing for responsive avoidance of obstacles.

The power consumption was 2.1 watts during straight-line movement and 3.4 watts during rotation caused by higher current draw by motors. With a battery size of 57.72Wh, there will be about 5 hours worth of battery power supply time.

B. Environmental Performance Factors

Performance is greatly influenced by environment properties. Straight corridors with regular geometry and many corner structures support accurate localization, whereas wide corridors with few structures pose problems for scan matching. Passage widths smaller than 1.5m limit sensor range, resulting in an approximate 18% decrease in mapping accuracy compared to wider spaces.

Moving objects cause time inconsistency in the occupancy map. Mirrors generate reflective signals of obstacles located about 15cm past the wall due to mirror-like glass reflections. Although these are evident on experimental maps, they did not impede robot navigation.
Surface properties influence odometry performance. Tiles with smooth textures induce slipping when the robot moves too fast, thereby introducing odometry error that must be corrected during scan matching. Experimentally, tiles with regularly spaced grout lines generated encoder error, which was later fixed using median filtering of encoder counts.

C. Identified System Limitations

The 2D LiDAR is unable to sense obstacles located outside of its scan plane. There will be risks of collision with obstacles such as tables, shelves, and overhanging structures that are not visible to the sensor. The use of additional sensors like depth cameras or 3D LiDAR is necessary in environments where there are significant non-floor obstacles.
Reactive obstacle avoidance does not have predictive power. Obstacles moving quickly might be able to get to the robot's path in time and cause collisions. Implementing trajectory prediction using machine learning might solve this problem.
Loop closure detection is based on geometric scans that cannot function effectively in symmetric environments with repeated structures. Loop closures may be inaccurate in corridors with regularly repeating doorways and create inaccurate maps. Using semantic labels or visual features may improve place recognition.
Odometry drift makes long-term mapping impossible without loop closures. In the absence of loop closures, odometry drift increases by about 0.28 cm per each meter of traversed distance. It means that only a small stretch of approximately 200-300 meters can be mapped at once.

VI. CONCLUSION

This paper introduced NAMI, an autonomous navigation system, which is a fully realized SLAM system. It was shown to perform simultaneously localization and mapping in practical indoor environments, where maps with stable occupancy grid could be generated, and navigation between two points could be autonomously executed.
Technical contributions of this paper include: systematic framework to integrate multi-sensors using LiDAR, IMU, and wheel odometry; precise calibration strategy to ensure accuracy in kinematic model; experimentation in real environments showing navigational capabilities; and analysis of failure cases.
It is shown by experiments that classical graph-based SLAM approaches can provide high-quality output when tuned properly. Over 85% success rates in navigating through structured corridors suggest its applicability for logistics purposes. Maps can be constructed using low-cost commercial sensors, and the maps are sufficiently consistent for autonomous navigation.
These identified weaknesses will provide a basis for future improvement opportunities. The incorporation of depth cameras will allow the detection of obstacles beyond the 2D scanning plane. Trajectory prediction through learning will help avoid moving obstacles. Visual semantic mapping will help with place recognition in symmetrical environments. Multi-robot coordination will make logistic applications possible.
The implementation of the autonomous robot shows that highly capable autonomous navigation does not necessarily involve costly proprietary devices and GPU-based computing. System design and tuning will allow efficient operation on limited-resource embedded systems. This project provides a starting point for exploring hybrid robot navigation approaches that combine classical SLAM and perception/decision-making learning techniques..

REFERENCES

[1] S. Thrun, W. Burgard, and D. Fox, "Probabilistic Robotics," MIT Press, 2005.

[2] G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, vol. 23, no. 1, pp. 34-46, 2007.

[3] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The Marathon 2: A Navigation System," IEEE/RSJ International Conference on Intelligent Robots and Systems, 2020.

[4] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," IEEE International Conference on Robotics and Automation, 2016.

[5] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, and W. Burgard, "g2o: A general framework for graph optimization," IEEE International Conference on Robotics and Automation, 2011.

[6] D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance," IEEE Robotics & Automation Magazine, vol. 4, no. 1, pp. 23-33, 1997.

[7] M. Quigley et al., "ROS: an open-source Robot Operating System," ICRA Workshop on Open Source Software, 2009.

[8] C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," IEEE Transactions on Robotics, vol. 32, no. 6, pp. 1309-1332, 2016.

[9] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” Foundations and Trends® in Robotics, vol. 6, no. 1-2, pp. 1–139, 2017.

[10] A. Ullah et al., “Simultaneous Localization and Mapping Based on Kalman Filter and Extended Kalman Filter,” Wireless Communications and Mobile Computing, vol. 2020, 2020 .